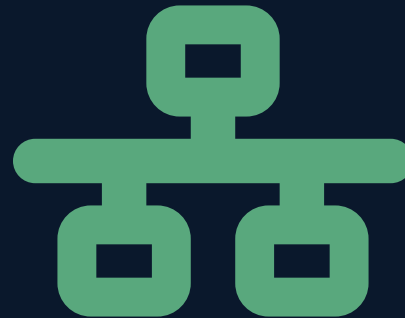


PORTADA

Arquitectura de Software

Semana 8 — Catálogo POSA: Microservicios, WebHooks

Universidad de Antioquía · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

3 días caídos.
Así empezó la migración de Netflix.



La migración de Netflix a microservicios

Empresa	Netflix
Motivador	Interrupción severa de base de datos en 2008
Duración de la caída	3 días sin poder enviar DVDs a los clientes
Decisión	Migrar el monolito a la nube (AWS) y descomponerlo en servicios independientes
Periodo de migración	2009 – 2012, de forma gradual, servicio por servicio
Resultado	Cientos de microservicios independientes en producción

Es uno de los casos de migración a microservicios más citados de la industria — Netflix documentó el proceso públicamente en su blog de ingeniería.

CASO · EL INCIDENTE DE 2008

Un único punto de falla tumbó todo el servicio

En agosto de 2008, una **corrupción en la base de datos central** de Netflix dejó a la empresa **3 días sin poder procesar ni enviar DVDs** a sus clientes — en ese momento, el corazón del negocio.

- El sistema completo dependía de una única base de datos monolítica.
- Un solo componente fallando bastó para detener toda la operación.
- No había forma de aislar el daño: si algo se rompía en el centro, todo dejaba de funcionar.

Ese incidente —no una moda tecnológica— fue el motivador directo detrás de la decisión de rediseñar la arquitectura.

CASO · LA DECISIÓN

De un monolito a cientos de servicios en la nube

Tras el incidente, Netflix decidió dos cosas al tiempo: mover su infraestructura a **AWS** (Amazon Web Services, un proveedor de nube pública) y **romper el monolito** en servicios pequeños e independientes.

- La migración fue **gradual**, entre 2009 y 2012, servicio por servicio, no un "big bang".
- Cada servicio nuevo se desplegaba, escalaba y fallaba **por su cuenta**, sin depender de que el resto del sistema estuviera "sano" al mismo tiempo.
- El objetivo explícito ya no era solo "funcionar", sino que **un fallo en un servicio no arrastrara a los demás**.

CASO · LA LECCIÓN CENTRAL

Aislar el fallo, no evitarlo por completo

La lección que Netflix repite en sus propios relatos técnicos no es "cómo evitar que algo falle" —eso es imposible a esa escala— sino **cómo evitar que la falla de una pieza tumbe el sistema completo**.

Aislamiento de fallos: diseñar el sistema para que el error de un componente quede contenido en ese componente, en vez de propagarse y derribar servicios que no tenían nada que ver con la causa original.

Con cientos de servicios independientes, si uno cae —o se satura— el resto de Netflix sigue funcionando: la recomendación de películas puede fallar sin que el video deje de reproducirse.

TRANSICIÓN

De la historia al patrón

Lo que Netflix construyó a partir de 2009 —sin usar todavía ese nombre— es hoy uno de los estilos arquitectónicos más usados de la industria: **Microservicios**. Formalicemos qué es exactamente y qué principios lo definen.

CONCEPTO

Microservicios: servicios pequeños, despliegue independiente

Microservicios es un estilo arquitectónico que descompone una aplicación en un conjunto de **servicios pequeños e independientemente desplegables**, cada uno responsable de una porción acotada del negocio.

- Cada servicio se puede **desplegar, actualizar y escalar por separado**, sin coordinar un release del sistema completo.
- Los servicios se comunican **por red** — HTTP/REST, mensajería asíncrona — nunca compartiendo memoria ni código en el mismo proceso.
- Cada servicio tiene, idealmente, **su propia base de datos**: nadie más accede a ella directamente.

CONCEPTO

Organizados por capacidad de negocio, no por capa técnica

La decisión de diseño más importante en microservicios es **cómo se traza el límite de cada servicio**: no se agrupan por capa técnica (todo el frontend junto, todo el acceso a datos junto), sino por **capacidad de negocio** (pagos, catálogo, recomendaciones, facturación).

Microservicios es, en buena medida, una **evolución de SOA** (Service-Oriented Architecture, arquitectura orientada a servicios): comparte la idea de dividir el sistema en servicios, pero pone mucho más énfasis en dos cosas que SOA no siempre exigía: **despliegue independiente real** y **autonomía total del equipo dueño del servicio** — ese equipo decide su stack, su base de datos y su ritmo de release.



CONCEPTO · VENTAJAS Y LIMITACIONES

Lo que Microservicios gana — y lo que cuesta

Ventajas	Limitaciones
Escalabilidad independiente: escalar solo el servicio que recibe más carga, sin escalar todo el sistema	Complejidad operativa masiva: decenas o cientos de servicios que desplegar, versionar y monitorear
Resiliencia: el fallo de un servicio no derriba al resto (aislamiento de fallos, como en el caso de Netflix)	Observabilidad difícil: entender qué pasó en un sistema distribuido exige logs, métricas y trazas centralizadas de todos los servicios
Equipos autónomos que despliegan a su propio ritmo, sin bloquear a otros equipos	Consistencia de datos distribuida: sin una única base de datos, mantener la información coherente entre servicios es un problema real
Cada servicio puede usar la tecnología más adecuada para su problema	Testing de integración más costoso: hay que probar cómo se comportan varios servicios funcionando juntos, no solo cada uno por separado

Observabilidad: capacidad de entender el estado interno de un sistema a partir de lo que expone hacia afuera (logs, métricas, trazas). Estos temas de complejidad operativa se retoman con detalle en las Unidades 3 y 4 del curso.

TRANSICIÓN

Servicios independientes que, aun así, deben avisarse

Si cada microservicio es autónomo y tiene su propia base de datos, ¿cómo se entera un servicio de que algo relevante ocurrió en otro? La respuesta obvia —**preguntar constantemente** ("¿ya cambió? ¿ya cambió?")— es ineficiente a escala.

Un patrón de comunicación distinto resuelve justo ese problema: el WebHook.

CASO · ORIGEN DEL TÉRMINO

Jeff Lindsay acuña "webhook", 2007

Término	"WebHook"
Autor	Jeff Lindsay
Año	2007
Dónde	Publicado en su blog personal
Idea original	Extender el concepto de "hook" de programación (un punto donde se puede "enganchar" código para que se ejecute cuando algo pasa) a la web

Hook (gancho): en programación, un punto de extensión donde se puede registrar código propio para que se ejecute automáticamente cuando ocurre cierto evento. Lindsay tomó esa idea y la llevó a peticiones HTTP entre sistemas web.

CASO · ADOPCIÓN MASIVA

De un post de blog a un estándar de facto

Lo que empezó como una idea en un blog personal se convirtió, con los años, en uno de los mecanismos de integración más usados entre sistemas web:

- **GitHub** usa webhooks para notificar a sistemas externos —como servidores de integración continua (CI/CD)— cada vez que hay un *push*, un *pull request* o un nuevo *issue*.
- **Stripe** usa webhooks para avisar a la aplicación del comerciante cuando ocurre un evento de pago: un cobro exitoso, un reembolso, una suscripción cancelada.

Ninguna de las dos plataformas inventó el patrón — lo adoptaron porque resolvía un problema real de comunicación entre sistemas independientes.



CONCEPTO

WebHook: notificación HTTP invertida

Un **WebHook** es un mecanismo donde, en vez de que el cliente pregunte repetidamente al servidor si algo cambió (*polling*), es el **servidor quien avisa al cliente** apenas ocurre el evento, mediante una petición HTTP POST a una URL que el cliente registró de antemano.

- **Polling:** el consumidor pregunta cada X segundos "¿ya hay algo nuevo?" — desperdicia peticiones cuando no hay nada que reportar.
- **WebHook:** el consumidor registra una URL ("callback") una sola vez; el servidor la llama únicamente cuando el evento realmente ocurre.

CONCEPTO

Cómo funciona, paso a paso

1. El consumidor registra una URL propia (**endpoint**) ante el servicio que emitirá el evento.
2. Ocurre el evento en el servicio origen (un pago, un *push* a un repositorio, un cambio de estado).
3. El servicio origen hace una petición **HTTP POST** a la URL registrada, con los datos del evento (**payload**) en el cuerpo de la petición.
4. El consumidor recibe la petición y reacciona de inmediato — sin haber tenido que preguntar nunca.



CONCEPTO · VENTAJAS Y LIMITACIONES

Lo que WebHooks gana — y lo que cuesta

Ventajas	Limitaciones
Evita el desperdicio de peticiones constantes del <i>polling</i>	El receptor necesita un endpoint público accesible desde internet, siempre disponible
Notificación casi en tiempo real , apenas ocurre el evento	El receptor debe manejar reintentos si su endpoint estuvo caído cuando llegó el evento
Reduce carga tanto en el emisor como en el consumidor comparado con preguntar todo el tiempo	Debe verificar la autenticidad de cada petición recibida — cualquiera podría intentar enviar un POST falso a esa URL

Microservicios y WebHooks se combinan con frecuencia: un servicio autónomo notifica a otros mediante webhooks (o patrones de eventos similares) en vez de que estos lo consulten por polling — comunicación asíncrona entre unidades independientes.

TENSIÓN CONCEPTUAL · CONTRAEJEMPLO

Un webhook sin validar es una puerta abierta

Un endpoint de webhook es, técnicamente, una URL pública que **acepta peticiones POST de cualquiera**. Si el receptor confía ciegamente en cualquier petición que le llega a esa URL, cualquiera puede simular eventos falsos: un "pago exitoso" que nunca ocurrió, un "push" que nadie hizo.

La mitigación estándar de la industria es la **firma HMAC**: el emisor firma cada petición con una clave secreta compartida; el receptor recalcula esa firma y solo procesa la petición si coincide.

HMAC (Hash-based Message Authentication Code): mecanismo criptográfico que permite verificar que un mensaje viene realmente de quien dice enviarlo y que no fue alterado en el camino.



INTERACCIÓN

Diagnosticuen el diseño

Para cada situación, identifiquen qué principio de hoy —microservicios o webhooks— está en riesgo y por qué:

1. Dos microservicios distintos leen y escriben directamente sobre la misma base de datos compartida.
2. Un sistema de facturación consulta cada 5 segundos si hubo un nuevo pago, en vez de esperar una notificación.
3. Un endpoint de webhook procesa cualquier petición POST que le llega, sin revisar de dónde viene ni si trae una firma válida.

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. **Microservicios** descompone el sistema en servicios pequeños, independientemente desplegables, con base de datos propia, organizados por capacidad de negocio — no por capa técnica.
2. El caso de **Netflix (2008-2012)** muestra el motivador real detrás de este estilo: un incidente de 3 días llevó a rediseñar el sistema para que el fallo de una pieza no tumbe el todo (aislamiento de fallos).
3. Microservicios gana escalabilidad y resiliencia independientes, a costa de complejidad operativa: observabilidad, consistencia distribuida y testing de integración.
4. **WebHooks** invierte la lógica de notificación: en vez de preguntar (*polling*), el servidor avisa al consumidor con un POST apenas ocurre el evento — término acuñado por Jeff Lindsay en 2007, adoptado hoy por GitHub, Stripe y muchas otras plataformas.
5. Un webhook sin validar (sin firma HMAC) es un riesgo de seguridad real: cualquiera puede simular eventos falsos hacia un endpoint público desprotegido.

Con Microservicios y WebHooks suman diez patrones del catálogo POSA vistos en la Unidad 2: Pipes & Filters, Blackboard, Broker, MVC, Batch, Cliente/Servidor N-capas, Layers, Microkernel, Microservicios y WebHooks.

CIERRE

Para la próxima semana

- Identificar, en el proyecto de Fábrica Escuela, si hoy existe algo parecido a un servicio independiente o a una notificación por evento, en vez de por consulta constante.
- Repasar la diferencia entre *polling* y notificación por webhook con un ejemplo propio, distinto a los vistos en clase.

La próxima sesión cierra el catálogo POSA de la Unidad 2 con tres patrones más: Backend for Frontend (BFF), Circuit Breaker y SOFEA.

